SD-DPC: Sparse Dictionary Differentiable Predictive Control

Ali Reza Daneshvar Garmroodi[1] and Jan Drgoňa[2]

***Abstract*— We present sparse dictionary differentiable predictive control (SD-DPC), a framework for learning sparse, interpretable feedback policies for nonlinear systems from data. A prediction model is first identified by rollout-based sparse identification of nonlinear dynamics (SINDy), building on gradient-based and multistep formulations. The policy is then parameterized as a sparse combination of dictionary functions and trained by differentiating a constrained finite-horizon predictive-control objective through this model, so that its terms are selected by closed-loop performance rather than by imitating a previously trained controller. The result is an explicit feedback law with only a handful of terms. Across three benchmark control problems, SD-DPC satisfies the constraints in almost all test scenarios, outperforms a policy distilled onto the same terms by up to an order of magnitude, and requires orders of magnitude less memory and online computation than an optimization benchmark, while admitting explicit sensitivity bounds.**

I. INTRODUCTION

Model predictive control (MPC) optimizes closed-loop performance under state and input constraints, but requires solving a constrained optimization problem at every sampling instant [1]–[3], which can be prohibitive for fast nonlinear systems or restricted hardware. Explicit MPC moves the optimization offline, although the number of regions describing the solution can grow rapidly with the problem dimension [4]. Learned policies scale better, either by approximating a precomputed MPC law [5]–[7] or by directly minimizing a predictive-control objective through differentiable closed-loop rollouts [8], [9]. The latter approach, differentiable predictive control (DPC), yields an explicit policy that is inexpensive to evaluate online. However, these learned policies are commonly deep neural networks, which distribute the control law across many coupled parameters and are difficult to interpret, verify, modify, and adapt after deployment [10].

Sparse identification of nonlinear dynamics (SINDy) represents system dynamics as a sparse combination of candidate functions [11]. Extensions to controlled systems enable compact models for model-based control [12], [13], while ensemble, weak-form, gradient-based, and trajectory-based formulations improve robustness to noise and avoid pointwise derivative estimates [14]–[18]. The same dictionary structure can also represent a feedback law, which is the idea pursued here.

*This research is supported by the U.S. DOE, Office of Science, ASCR program under the Scientific Discovery through Advanced Computing (SciDAC) Institute "LEADS: LEarning-Accelerated Domain Science".

[1]Concordia University, Montreal, QC H3G1M8, Canada. alireza.daneshvargarmroodi@mail.concordia.ca

[2]Johns Hopkins University, Baltimore, Maryland 21218, USA. jdrgona1@jh.edu

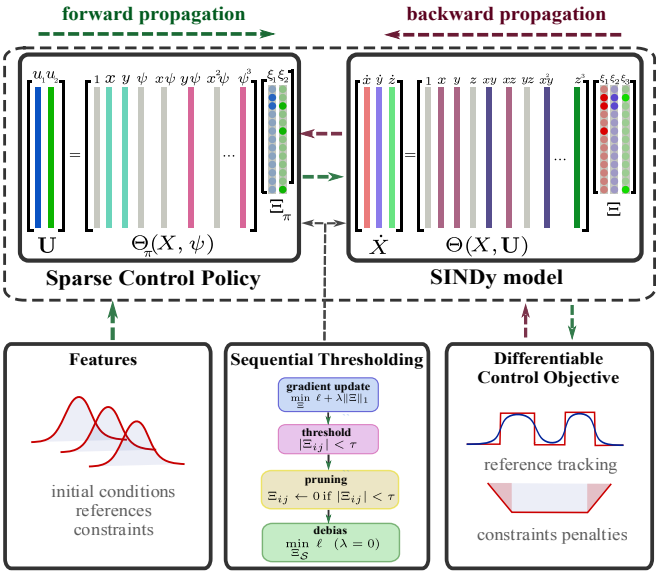


Fig. 1. Pipeline: sparse system identification and differentiable predictive control policy learning.

Sparse and symbolic representations have also been used for interpretable control policies. One line of work optimizes the policy structure through model-free reinforcement learning [19]–[21], where constraints enter only through the reward. Another trains a neural policy first and then fits a sparse policy to its actions [22], so the result is selected for agreement with the teacher rather than for closed-loop performance. Sparse polynomial feedback laws have also been learned by minimizing an averaged closed-loop cost with an ℓ_1 penalty [23], but for known control-affine dynamics with a constant input matrix and without state or input constraints. To our knowledge, the terms of an explicit sparse policy have not been selected under a constrained finite-horizon predictive-control objective evaluated through a model identified from data.

In this work, we train a sparse dictionary policy directly through DPC. The prediction model and the feedback law are both sparse combinations of candidate functions, learned by the same sequentially thresholded gradient procedure, so the policy terms are selected by the constrained closed-loop objective itself rather than by an intermediate imitation step. The contributions of this paper are as follows.

- **Closed-loop sparse policy learning framework:** SD-DPC represents both the prediction model and the feedback policy as sparse dictionary expansions and, to our knowledge, is the first to select the policy terms under a constrained predictive-control objective through a model identified from data [16]–[18].

- **Structural guarantees:** we show finite termination of the thresholded training with a bound on the cost of each prune, and derive a Lipschitz bound of the policy from its retained coefficients that bounds the closed-loop deviation caused by pruning or model mismatch.
- **Closed-loop validation:** on three benchmarks evaluated on the true plants, SD-DPC satisfies almost all constraints with orders of magnitude less memory and evaluation time than online optimization, whereas distilling a deep neural policy onto the same terms increases the tracking error by up to an order of magnitude and violates the obstacle constraint.

The remainder of the paper is organized as follows. Section II reviews SINDy and DPC, Section III develops SD-DPC, its training algorithm, and its structural properties, Section IV details the case studies, Section V presents the results, and Section VI concludes.

II. Background

A. Sparse Identification of Nonlinear Dynamics

Consider the nonlinear controlled system $\dot{x}(t) = f(x(t), u(t))$ with state $x \in \mathbb{R}^{n_x}$ and input $u \in \mathbb{R}^{n_u}$, where f is unknown but trajectory data are available. Let $X \in \mathbb{R}^{n_s \times n_x}$ and $U \in \mathbb{R}^{n_s \times n_u}$ collect n_s state and input samples, and let $\dot{X} \in \mathbb{R}^{n_s \times n_x}$ contain the corresponding state derivatives, measured or estimated by numerical differentiation. SINDy approximates f by a small number of candidate functions collected in a library

$$\Theta(X, U) = \big[\theta_1(X, U) \ \cdots \ \theta_p(X, U)\big] \in \mathbb{R}^{n_s \times p}, \quad (1)$$

typically polynomial, trigonometric, and state–input interaction terms, so that $\dot{X} \approx \Theta(X, U)\Xi$ with $\Xi \in \mathbb{R}^{p \times n_x}$. The coefficients follow from the sparsity-promoting regression

$$\min_{\Xi} \ \tfrac{1}{2}\big\|\dot{X} - \Theta(X, U)\Xi\big\|_F^2 + \lambda\, \mathcal{R}(\Xi), \quad (2)$$

where $\mathcal{R}$ promotes sparsity and $\lambda > 0$ weights it. In the standard formulation this is solved by sequentially thresholded least squares: a least-squares fit, removal of coefficients below a threshold, and refitting on the retained terms, repeated until the retained coefficient set no longer changes.

B. Differentiable Predictive Control

DPC learns an explicit feedback law offline by treating the constrained optimal-control problem as an unsupervised learning problem [8], [9]. The prediction model is held fixed while the policy parameters are optimized through finite-horizon closed-loop rollouts. Let F denote the discrete-time prediction map induced by the model and the integration scheme, and let $\psi_t \in \Psi^{\mathrm{ctrl}}$ denote the control features available at time t, such as reference signals or admissible bounds. The policy parameters w are obtained by solving

$$\min_{w} \quad \frac{1}{m}\sum_{j=1}^{m}\Big(\sum_{t=1}^{N} \ell_{\mathrm{DPC}}(x_t, u_{t-1}, \psi_t) + \ell_N(x_N, \psi_N)\Big) \quad (3a)$$

$$\text{s.t.} \quad x_{t+1} = F(x_t, u_t), \qquad u_t = \pi_w(x_t, \psi_t), \quad (3b)$$

$$x_t \in \mathcal{X}, \quad u_t \in \mathcal{U}, \quad \psi_t \in \Psi^{\mathrm{ctrl}}, \quad (3c)$$

over m sampled initial conditions and reference realizations, with $t = 0, \dots, N-1$ in (3b). The stage loss ℓ_{DPC} collects the tracking objective, control regularization, and differentiable penalties on constraint violation; in practice, the constraints (3c) are enforced only through these penalties. It may also penalize the increments $x_t - x_{t-1}$ and $u_{t-1} - u_{t-2}$, with the convention $u_{-1} := u_0$, and ℓ_N is an optional terminal loss. Since the rollout is differentiable in w, gradients propagate through the entire horizon, so each action is optimized with its downstream effect taken into account. Formulation (3) leaves open how F is obtained and how π_w is parameterized; Section III addresses both.

III. Sparse Dictionary Differentiable Predictive Control

SD-DPC proceeds in two stages, both instances of the thresholded gradient-descent procedure of Algorithm 1 applied to different losses. A sparse dynamics model is first identified from multistep rollout error, providing the differentiable prediction map; a sparse dictionary policy is then trained by differentiating a finite-horizon closed-loop objective through that map. Fig. 1 summarizes the pipeline.

A. Rollout-Based Sparse Identification

We estimate Ξ from rollout errors of the SINDy model rather than from state-derivative data, so that $\dot{X}$ never enters the objective and the identification does not inherit numerical differentiation error. Introducing the one-step map induced by a fixed-step numerical integrator with step Δt and zero-order hold on the input,

$$x_{k+1} = \mathrm{ODESolve}\big(\hat{f},\ x_k,\ u_k,\ \Delta t\big), \quad (4)$$

which we implement with the fourth-order Runge–Kutta scheme for the continuous-time examples and use consistently during identification and policy training. The data consist of m_{id} trajectories $\{(\hat{x}_0^j, \hat{u}_0^j), \dots, (\hat{x}_{N_{\mathrm{id}}}^j, \hat{u}_{N_{\mathrm{id}}}^j)\}_{j=1}^{m_{\mathrm{id}}}$ collected under persistently exciting inputs, with N_{id} the identification horizon. Applying the measured inputs to the model yields the predicted states $\{x_k^j\}$, and the coefficients are obtained from

$$\min_{\Xi} \quad \frac{1}{m_{\mathrm{id}}}\sum_{j=1}^{m_{\mathrm{id}}}\Big(Q_1\big\|x_1^j - \hat{x}_1^j\big\|_2^2 + Q_N\sum_{k=2}^{N_{\mathrm{id}}}\big\|x_k^j - \hat{x}_k^j\big\|_2^2\Big) + \ \lambda\|\Xi\|_1 \quad (5a)$$

$$\text{s.t.} \quad x_{k+1}^j = \mathrm{ODESolve}\big(\hat{f},\ x_k^j,\ \hat{u}_k^j,\ \Delta t\big), \quad (5b)$$

$$x_0^j = \hat{x}_0^j, \quad (5c)$$

where $\lambda > 0$ is the sparsity weight and $Q_1, Q_N \geq 0$ weight the one-step and multistep prediction errors. With $N_{\mathrm{id}} = 1$ the multistep sum is empty, and an explicit Euler step gives $x_1^j = \hat{x}_0^j + \Delta t\, \hat{f}(\hat{x}_0^j, \hat{u}_0^j)$, which is linear in Ξ, so (5) reduces to the ℓ_1-regularized SINDy regression with forward-difference derivatives [11]; the multistep terms extend it by penalizing the error accumulated along the rollout.

B. Differentiable Sparse Dictionary Policy Parameterization

The feedback policy uses the same dictionary structure. With $\mathcal{X} \subset \mathbb{R}^{n_x}$ and $\mathcal{U} \subset \mathbb{R}^{n_u}$ the admissible state and input

sets, the policy input is $z_t = [x_t \ \psi_t]$ and the policy library is

$$\Theta_\pi(z) = \big[1,\ z^\top,\ \phi_{\mathrm{pol}}(z)^\top,\ \phi_{\mathrm{tr}}(z)^\top\big] \in \mathbb{R}^{1\times p_\pi}, \tag{6}$$

where ϕ_{pol} collects quadratic and cross terms and ϕ_{tr} trigonometric terms, with maximal polynomial degree and trigonometric frequency treated as hyperparameters. The sparse dictionary policy is

$$u_t = \pi_{\Xi_\pi}(z_t) = \Pi_{\mathcal{U}}\big(\Theta_\pi(z_t)\,\Xi_\pi\big), \qquad \Xi_\pi \in \mathbb{R}^{p_\pi\times n_u}, \tag{7}$$

with $\Pi_{\mathcal{U}}$ the componentwise projection enforcing $\underline{u} \le u_t \le \overline{u}$. Column i of Ξ_π feeds input channel i alone, so sparsity in Ξ_π determines directly which feedback terms are retained in each channel, and the policy is evaluated without any online optimization. The exact derivative of the piecewise-affine projection $\Pi_{\mathcal{U}}$ annihilates the sensitivity of every saturated channel, so a differentiation surrogate is required. With the raw channel output $v_i(z) := [\Theta_\pi(z)\Xi_\pi]_i$, a band half-width $b_i \ge 0$, a leak $\epsilon \in [0,1]$, and the band $\mathcal{V}_i := [\underline{u}_i - b_i,\ \overline{u}_i + b_i]$, the forward pass evaluates (7) exactly, so every loss and reported trajectory is that of the true saturated closed loop, while the backward pass uses the surrogate derivative

$$\Gamma^\epsilon := \mathrm{diag}(\gamma_1^\epsilon, \dots, \gamma_{n_u}^\epsilon), \qquad \gamma_i^\epsilon := \begin{cases} 1, & v_i \in \mathcal{V}_i, \\ \epsilon, & v_i \notin \mathcal{V}_i, \end{cases} \tag{8}$$

which recovers the a.e. derivative of $\Pi_{\mathcal{U}}$ at $(\epsilon, b) = (0, 0)$ and the plain straight-through estimator [24] at $(\epsilon, b) = (1, 0)$. The intermediate regime avoids two failure modes. With $\epsilon = 0$, a coefficient feeding a saturated channel receives no gradient from the loss and only shrinks, so thresholding acts on stale rather than fitted evidence; with $\epsilon = 1$, a deeply saturated channel exerts influence through a sensitivity the plant does not possess. The leak attenuates channels far outside $\mathcal{U}$ without silencing them, while a small band b keeps the gradient from switching abruptly as a channel oscillates about its bound.

C. SD-DPC Problem Formulation and Training Algorithm

Adding a sparsity penalty and using the sparse dictionary policy in (3) gives

$$\min_{\Xi_\pi} \quad \frac{1}{m}\sum_{j=1}^{m}\Big(\sum_{t=1}^{N}\ell_{\mathrm{DPC}}(x_t, u_{t-1}, \psi_t) + \ell_N(x_N, \psi_N)\Big) + \lambda_\pi\|\Xi_\pi\|_1 \tag{9a}$$

$$\text{s.t.} \quad x_{t+1} = F_\Xi(x_t, u_t), \qquad u_t = \pi_{\Xi_\pi}(z_t), \tag{9b}$$

$$x_0 \sim \mathcal{P}_{x_0}, \quad \psi_t \in \Psi^{\mathrm{ctrl}}, \tag{9c}$$

where F_Ξ is the prediction map (4) of the identified SINDy model. The input constraint $u_t \in \mathcal{U}$ holds by construction through $\Pi_{\mathcal{U}}$, whereas the state constraint $x_t \in \mathcal{X}$ enters ℓ_{DPC} through penalty terms, specified for each example in Section IV. Both problems have the composite form of a loss ℓ, differentiable almost everywhere, plus an entrywise ℓ_1 penalty. Their coefficients are updated by a first-order method, using a subgradient for the ℓ_1 term, and are periodically pruned, as in sequentially thresholded gradient training of SINDy models [16]; the two stages differ only in the loss and in the matrix being sparsified (Algorithm 1). The entrywise penalty $\|\Xi\|_1 = \sum_{i,j}|\Xi_{ij}|$ shrinks uninformative coefficients toward zero without setting them exactly to zero, so a separate thresholding step is required. Every n_p epochs the retained coefficient set is updated entrywise,

$$\mathcal{S} = \{\,(i,j)\ :\ |\Xi_{ij}| \ge \tau\,\}, \tag{10}$$

so each output channel j retains its own subset of dictionary terms. Entries outside $\mathcal{S}$ are set to zero and excluded from subsequent updates, a library column is dropped from the compiled evaluation once its term is removed from every channel, and the optimizer is reinitialized on the reduced parameter set. The procedure stops once $\mathcal{S}$ has been stable over M consecutive prunes or the epoch budget is exhausted, after which the coefficients in $\mathcal{S}$ are refitted without the ℓ_1 penalty to remove its shrinkage bias. In our implementation, the update in line 3 is carried out with AdamW [25]; the properties below concern the pruning mechanism and do not depend on this choice.

Algorithm 1 Sequentially Thresholded Gradient Descent with Debiasing

Require: Θ: library of p terms; ℓ: loss (a.e. differentiable)
$\Xi \in \mathbb{R}^{p\times n}$: coefficients, Ξ_{ij} = term i, channel j
τ: threshold; n_p: prune interval; n_e: epochs; M: patience; K: debias epochs
Note: applied to (5) for Ξ ($n = n_x$), and to (9) for Ξ_π ($n = n_u$, $(\lambda, \tau) \to (\lambda_\pi, \tau_\pi)$) with $\partial\Pi_{\mathcal{U}} = \Gamma^\epsilon$ of (8).

1: $\mathcal{S}_{\mathrm{prev}} \leftarrow$ all entries of Ξ; $\quad s \leftarrow 0$
2: **for** $e = 1, \dots, n_e$ **do**
3: Update $\Xi_{\mathcal{S}_{\mathrm{prev}}}$ by minimizing $\ell + \lambda\|\Xi\|_1$
4: **if** $e \bmod n_p = 0$ **then**
5: $\mathcal{S} \leftarrow \{(i,j) : |\Xi_{ij}| \ge \tau\}$ ▷ per-channel
6: **if** $\mathcal{S} \ne \mathcal{S}_{\mathrm{prev}}$ **then**
7: $\Xi_{ij} \leftarrow 0$ for all $(i,j) \notin \mathcal{S}$
8: Drop from Θ the terms removed from every channel
9: Reinitialize optimizer on the reduced set
10: $\mathcal{S}_{\mathrm{prev}} \leftarrow \mathcal{S}$; $\quad s \leftarrow 0$
11: **else**
12: $s \leftarrow s + 1$; $\quad$ **if** $s \ge M$ **then break**
13: **end if**
14: **end if**
15: **end for**
16: Save the biased sparse coefficients $\Xi^{\mathrm{bias}} \leftarrow \Xi$
17: Warm-start $\Xi_{\mathcal{S}}$ from Ξ^{bias} and reinitialize the optimizer
18: **for** $e = 1, \dots, K$ **do**
19: Update only $\Xi_{\mathcal{S}}$ by minimizing ℓ with $\lambda = 0$ ▷ fixed retained set
20: **end for**
21: **return** debiased sparse coefficient matrix Ξ

Proposition 1 (Finite support stabilization)**.** *Let $\tau > 0$. In Algorithm 1, a coefficient that has been pruned is never retained again. Consequently, the retained set changes at most pn times, and the thresholding loop terminates after at most $n_p M(pn+1)$ epochs, for any loss and any epoch budget n_e.*

Proof. A pruned coefficient is set to zero and excluded from all subsequent updates, so its value remains zero. Since $\tau > 0$, it fails the test $|\Xi_{ij}| \ge \tau$ at every later prune and is never retained again. Hence, if $\mathcal{S}_r$ denotes the retained set after the r-th change, each change removes at least one entry, and

$\mathcal{S}_{r+1} \subsetneq \mathcal{S}_r$. As Ξ has pn entries, at most pn such changes can occur. The loop stops after M consecutive prunes without a change, so each change is preceded by at most $M-1$ unchanged prunes, and the last change is followed by exactly M. The total number of prunes is therefore at most $M(pn+1)$, and since prunes are n_p epochs apart, the loop terminates within $n_p M(pn+1)$ epochs. □ □

Proposition 1 splits Algorithm 1 into a finite phase that selects which terms are retained and a refit that only adjusts their values. Since pruning occurs before the optimization has converged, its cost is bounded through a stationarity residual.

Proposition 2 (Pruning cost)**.** *Let $J = \ell + \lambda\|\cdot\|_1$, let Ξ' be obtained from Ξ by zeroing a set $\mathcal{D}$ of q retained entries with $0 < |\Xi_{ij}| < \tau$, let $\nabla\ell$ be β-Lipschitz on the segment between them, and let the residuals $r_{ij} = \partial\ell(\Xi)/\partial\Xi_{ij} + \lambda\,\mathrm{sign}(\Xi_{ij})$ on $\mathcal{D}$ satisfy $\|r_{\mathcal{D}}\|_F \le \eta$. Then $J(\Xi') \le J(\Xi) + \eta\tau\sqrt{q} + \frac{\beta}{2}q\tau^2$ and $\ell(\Xi') \le \ell(\Xi) + \lambda q\tau + \eta\tau\sqrt{q} + \frac{\beta}{2}q\tau^2$.*

Proof. Let $\Delta = \Xi' - \Xi$, so that $\|\Delta\|_F \le \tau\sqrt{q}$. The descent lemma and $\lambda\|\Xi'\|_1 - \lambda\|\Xi\|_1 = \langle \lambda\,\mathrm{sign}(\Xi), \Delta\rangle$ give $J(\Xi') - J(\Xi) \le \langle r_{\mathcal{D}}, \Delta_{\mathcal{D}}\rangle + \frac{\beta}{2}\|\Delta\|_F^2$, and the Cauchy–Schwarz inequality yields the first bound; the second adds $\lambda\sum_{\mathcal{D}}|\Xi_{ij}| \le \lambda q\tau$. □ □

The residual vanishes at a stationary point and otherwise absorbs the optimizer's inexactness and the surrogate bias (8); the bound on ℓ shows why λ and τ are tuned jointly (Fig. 2). In the policy stage, smoothness holds along segments on which no channel crosses a saturation or constraint boundary.

D. Structural Properties of SD-DPC Closed Loops

Since the learned policy is a finite sum of known functions, its sensitivity can be bounded explicitly from the retained terms. We write $\pi = \Pi_{\mathcal{U}}(v)$ with the raw output $v(x,\psi) = \Theta_\pi(x,\psi)\Xi_\pi$, let $\mathcal{S}_j$ collect the terms retained in channel j, and let $\|\cdot\|$ denote the Euclidean norm.

Assumption 1. States and features lie in compact sets $\tilde{\mathcal{X}}$ and $\tilde{\Psi}$, on which (a) each library function $\theta_{\pi,i}(\cdot,\psi)$ is L_i-Lipschitz for every $\psi \in \tilde{\Psi}$; (b) $\|\Theta_\pi(x,\psi)\| \le B_\Theta$; and (c) $\|F(x,u) - F(x',u')\| \le L_x\|x-x'\| + L_u\|u-u'\|$ for all $x, x' \in \tilde{\mathcal{X}}$ and $u, u' \in \mathcal{U}$.

Proposition 3 (Lipschitz bound and perturbation of sparse policies)**.** *Let Assumption 1 hold. Then, for all $x, x' \in \tilde{\mathcal{X}}$ and $\psi \in \tilde{\Psi}$:*

(i) Lipschitz bound: $\|\pi(x,\psi) - \pi(x',\psi)\| \le L_\pi\|x-x'\|$, *with*

$$L_\pi := \Big(\sum_{j=1}^{n_u}\big(\sum_{i\in\mathcal{S}_j}|\Xi_{\pi,ij}|\,L_i\big)^2\Big)^{1/2}. \tag{11}$$

(ii) Policy perturbation: *the policy $\tilde{\pi} = \Pi_{\mathcal{U}}(\Theta_\pi\tilde{\Xi}_\pi)$ with perturbed coefficients satisfies $\|\pi(x,\psi) - \tilde{\pi}(x,\psi)\| \le B_\Theta\|\Xi_\pi - \tilde{\Xi}_\pi\|_F$; in particular, pruning q coefficients below τ_π changes the action by at most $B_\Theta\tau_\pi\sqrt{q}$.*

(iii) Trajectory perturbation: *let $\tilde{\pi}$ be any policy with values in $\mathcal{U}$ and $\|\pi - \tilde{\pi}\| \le \varepsilon_\pi$ on $\tilde{\mathcal{X}} \times \tilde{\Psi}$, let $\|F - \tilde{F}\| \le \delta$ on $\tilde{\mathcal{X}} \times \mathcal{U}$, and let $x_{t+1} = F(x_t, \pi(x_t,\psi_t))$ and $\tilde{x}_{t+1} = \tilde{F}(\tilde{x}_t, \tilde{\pi}(\tilde{x}_t,\psi_t))$ start from the same x_0 under the same $\psi_t \in \tilde{\Psi}$ and remain in $\tilde{\mathcal{X}}$ for $t \le N$. Then*

$$\|x_t - \tilde{x}_t\| \le e_t := (\delta + L_u\varepsilon_\pi)\textstyle\sum_{s=0}^{t-1} a^s \tag{12}$$

for $t \le N$, where $a := L_x + L_u L_\pi$. Moreover, if h is L_h-Lipschitz on $\tilde{\mathcal{X}}$ and one of the two trajectories satisfies $h \ge L_h e_t$ for $t \le N$, the other satisfies $h \ge 0$.

Proof. (i) The componentwise projection $\Pi_{\mathcal{U}}$ is 1-Lipschitz, so $\|\pi(x,\psi) - \pi(x',\psi)\| \le \|v(x,\psi) - v(x',\psi)\|$. By Assumption 1(a), $|v_j(x,\psi) - v_j(x',\psi)| \le \sum_{i\in\mathcal{S}_j}|\Xi_{\pi,ij}|\,|\theta_{\pi,i}(x,\psi) - \theta_{\pi,i}(x',\psi)| \le \big(\sum_{i\in\mathcal{S}_j}|\Xi_{\pi,ij}|L_i\big)\|x-x'\|$, and summing the squares over j gives (i). (ii) Likewise, $\|\pi - \tilde{\pi}\| \le \|\Theta_\pi(\Xi_\pi - \tilde{\Xi}_\pi)\| \le \|\Theta_\pi\|\,\|\Xi_\pi - \tilde{\Xi}_\pi\|_F$, and q removed entries below τ_π have Frobenius norm at most $\tau_\pi\sqrt{q}$. (iii) With $d_t := \|x_t - \tilde{x}_t\|$ and ψ_t omitted, split $x_{t+1} - \tilde{x}_{t+1}$ into $F(x_t,\pi(x_t)) - F(\tilde{x}_t,\pi(\tilde{x}_t))$, $F(\tilde{x}_t,\pi(\tilde{x}_t)) - F(\tilde{x}_t,\tilde{\pi}(\tilde{x}_t))$, and $F(\tilde{x}_t,\tilde{\pi}(\tilde{x}_t)) - \tilde{F}(\tilde{x}_t,\tilde{\pi}(\tilde{x}_t))$. By Assumption 1(c) and (i), these are bounded by $a\,d_t$, $L_u\varepsilon_\pi$, and δ, so $d_{t+1} \le a\,d_t + L_u\varepsilon_\pi + \delta$ with $d_0 = 0$, and induction gives (12). Finally, $|h(x_t) - h(\tilde{x}_t)| \le L_h d_t \le L_h e_t$. □ □

All constants are explicit: $L_i = \omega$ for $\sin\omega x_k$ and $\cos\omega x_k$, $L_i = 0$ for terms in ψ alone, and, on a convex $\tilde{\mathcal{X}}$, L_i is the largest gradient norm of $\theta_{\pi,i}$. Since $L_\pi \le (\max_i L_i)\|\Xi_\pi\|_1$, the ℓ_1 penalty also limits the sensitivity. The model constants L_x and L_u follow from Ξ and the integrator in the same way, so Proposition 3 is evaluated from the two sparse coefficient matrices and the library, whereas for deep neural policies such bounds require products of layer norms or semidefinite relaxations [26]. By (ii) and (iii), a pruning step ($\tilde{F} = F$) or a model error with $\tilde{\pi} = \pi$, such as the transfer from F_Ξ to the plant, moves the trajectory in proportion to its size, with growth at most geometric at rate a; since typically $a > 1$, the bound is conservative over long horizons, and $\tilde{\mathcal{X}}$ is chosen to contain both trajectories (Section V).

IV. Numerical Examples

Three numerical case studies demonstrate the flexibility of SD-DPC: constrained reference tracking of a nonlinear system, stabilization of a nonlinear oscillator at its unstable origin, and an obstacle-avoidance reaching task. All examples are implemented in NeuroMANCER v1.5.3 [27], and all computations are performed on a MacBook with an Apple M1 processor and 8 GB of RAM. The code is available at `https://github.com/SOLARIS-JHU/SD-DPC`.

In each case study, SD-DPC is compared against three baselines. The first is an optimization-based benchmark solved with IPOPT [28] through CasADi [29], which minimizes the same stage and terminal costs as the policy objective, unless stated otherwise in the case study, and uses the true dynamics, and therefore has exact model knowledge. For the two-tank system, it is a receding-horizon MPC whose horizon starts at 50 steps, shrinks toward one step as the end of each reference segment approaches, and is reset when the reference changes. For the Van der Pol oscillator and the 2-D integrator, the full N-step problem is solved once per

scenario and the optimal input sequence is applied in open loop. Because this benchmark is computed for each scenario separately, whereas a learned policy is a single feedback law shared by all scenarios, it serves as a performance reference rather than as a competitor. NN-DPC is a deep neural policy trained by (3) on the same identified model and represents the standard DPC parameterization; in all cases, it is a multilayer perceptron with two hidden layers of 32 units and GELU activations. Finally, NN-Distilled follows the distillation approach of SINDy-RL [22]: a sparse dictionary policy is regressed onto the state–action pairs generated by the trained NN-DPC policy. Its terms are fixed to those retained by SD-DPC, so the two differ only in how the coefficients on this common support are obtained, either by regression onto the neural policy's actions or under the closed-loop objective (9), which isolates the effect of the training objective.

All policies are trained on rollouts of the identified model F_Ξ, whereas all controllers are evaluated on the true plants (13), (15), and (17), so that the reported results include the effect of model mismatch. Since state constraints, including the obstacle, are only penalized, their satisfaction is assessed on the evaluation scenarios of Section V.

A. Two-Tank System

We consider a cascaded two-tank system in which a pump feeds an upper tank that drains by gravity into a lower one. The state $x = [x_1, x_2]^\top$ collects the two liquid levels and the inputs $u = [u_1, u_2]^\top$ are the pump command and the valve split,

$$\dot{x}_1 = c_1(1-u_2)u_1 - c_2\sqrt{x_1}, \tag{13a}$$
$$\dot{x}_2 = c_1 u_2 u_1 + c_2\sqrt{x_1} - c_2\sqrt{x_2}, \tag{13b}$$

with inlet and outlet valve coefficients $(c_1, c_2) = (0.08, 0.04)$. The policy drives the levels to time-varying references r_t within the level bounds $\underline{x} = 0$ and $\overline{x} = 1$, with inputs in $\mathcal{U} = [0,1]^2$ and features $\psi_t = (r_t, \underline{x}_t, \overline{x}_t)$. The stage loss combines reference tracking with penalties on the level bounds,

$$\ell_{\mathrm{DPC}} = Q_r\|r_t - x_t\|_2^2 + \ell_{\underline{x},t} + \ell_{\overline{x},t},$$
$$\ell_{\underline{x},t} = Q_x\big\|[\underline{x} - x_t]_+\big\|_2^2, \quad \ell_{\overline{x},t} = Q_x\big\|[x_t - \overline{x}]_+\big\|_2^2, \tag{14}$$

where $[z]_+ = \max(0, z)$, and there is no terminal loss, $\ell_N \equiv 0$.

B. Forced Van der Pol Oscillator

We next consider the forced Van der Pol oscillator, whose uncontrolled trajectories converge to an attracting limit cycle surrounding an unstable origin. With $x = [x_1, x_2]^\top$ the position and velocity and u the forcing,

$$\dot{x}_1 = x_2, \qquad \dot{x}_2 = \mu(1-x_1^2)x_2 - x_1 + u, \qquad \mu = 1. \tag{15}$$

The policy regulates the state to the origin, thereby suppressing the limit cycle, with input bounds $\overline{u} = -\underline{u} = 5$ and state bounds $\overline{x} = -\underline{x} = [5,5]^\top$. Since the target is the fixed origin rather than an exogenous signal, the policy is pure state feedback, $z_t = x_t$. The stage loss penalizes the state, its increments, and the input increments, together with the bound penalties of (14),

$$\ell_{\mathrm{DPC}} = Q_r\|x_t\|_2^2 + Q_u\|u_{t-1}\|_2^2 + Q_{du}\|u_{t-1} - u_{t-2}\|_2^2 + \ell_{\underline{x},t} + \ell_{\overline{x},t}. \tag{16}$$

and $\ell_N \equiv 0$. Initial conditions are sampled both on and off the limit cycle so that the coefficients are not specialized to one region of the phase plane.

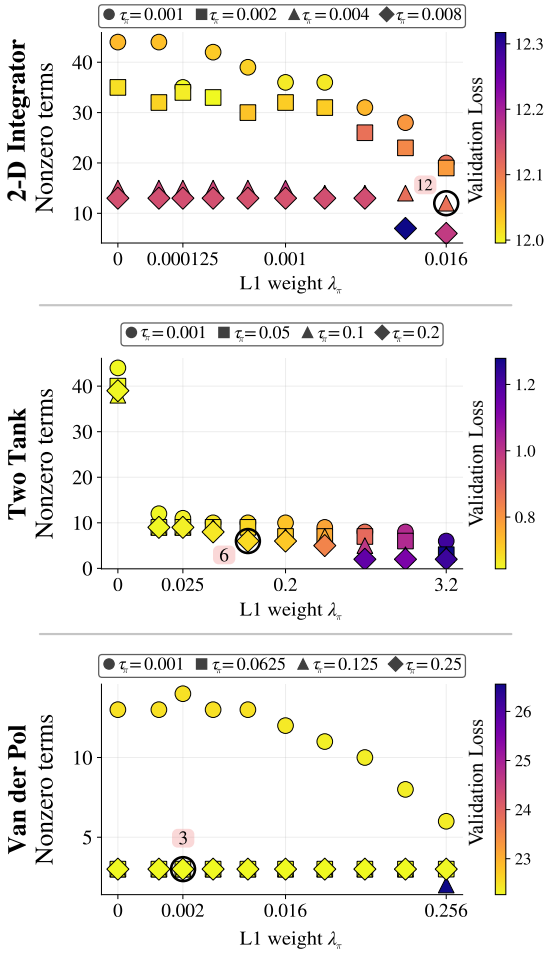


Fig. 2. Policy sparsity versus ℓ_1 weight λ_π. Marker shape denotes pruning threshold τ_π, color shows the loss (excluding the ℓ_1 penalty) on the validation scenarios used for selection, and black circles indicate the selected policies.

C. Two-Dimensional Integrator with Obstacle Avoidance

The third case study demonstrates handling of nonlinear state constraints. We consider a discrete-time integrator in the plane, in which each input sets the increment of the corresponding position coordinate,

$$x_{t+1} = A\,x_t + B\,u_t, \qquad A = \begin{bmatrix}1 & 0\\ 0 & 1\end{bmatrix}, \quad B = \begin{bmatrix}1 & 0\\ 0 & 1\end{bmatrix}, \tag{17}$$

with $x \in \mathbb{R}^2$ and $u \in \mathbb{R}^2$. The dynamics are linear and decoupled, so the difficulty of this example lies entirely in the constraint, which excludes a region of the state space,

$$h_{\mathrm{obs}}(x) = b_o\,(x_1 - x_1^o)^2 + (x_2 - x_2^o)^2 - (p_o/2)^2 \;\geq\; 0, \tag{18}$$

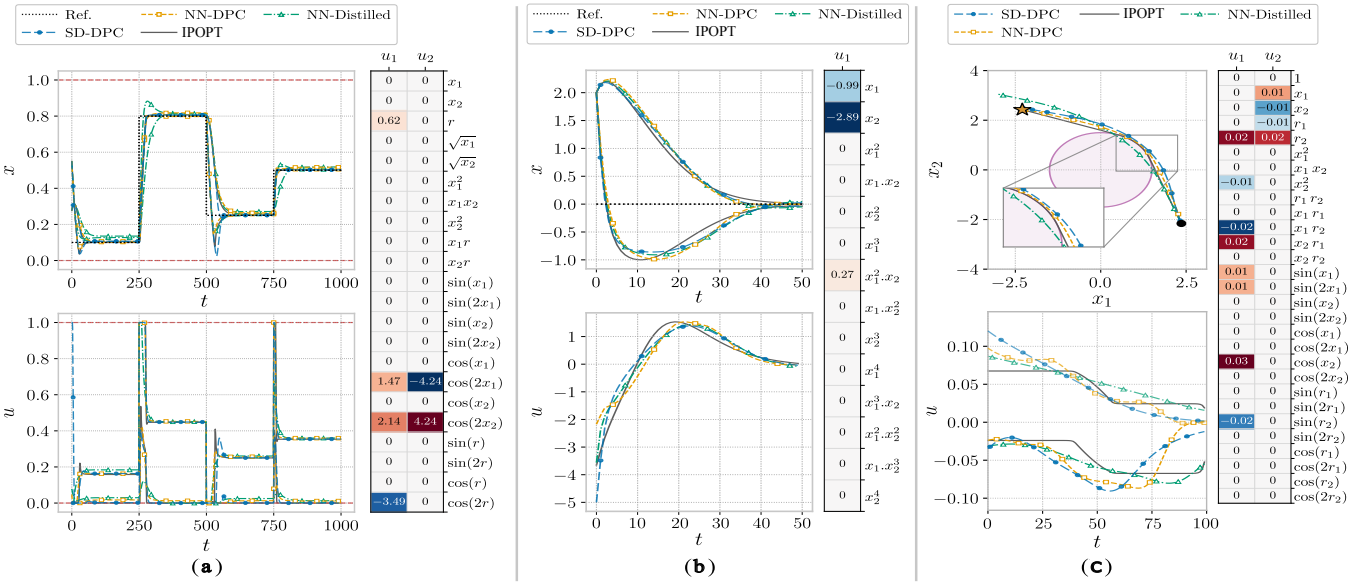


Fig. 3. Closed-loop trajectories of SD-DPC, NN-DPC, NN-Distilled, and the IPOPT benchmark, with the retained SD-DPC policy coefficients, for (a) the two-tank system, (b) the forced Van der Pol oscillator, and (c) the 2-D integrator with obstacle avoidance.

where p_o, b_o, and $x^o = (x_1^o, x_2^o)$ set the size, shape and center of the obstacle, here $p_o = 3$, $b_o = 1$, and $x^o = (0, 0)$, i.e., a circle of radius 1.5 centered at the origin. The objective is to reach a terminal setpoint r_N from initial conditions placed so that the direct path is blocked, with inputs bounded by $\underline{u} \le u_t \le \overline{u}$. The stage and terminal losses are

$$\begin{aligned}\ell_{\mathrm{DPC}} &= Q_{dx}\|x_t - x_{t-1}\|^2 + Q_u\|u_{t-1}\|^2 \\ &\quad + Q_{du}\|u_{t-1} - u_{t-2}\|^2 + Q_h\big[-h_{\mathrm{obs}}(x_t)\big]_+, \\ \ell_N &= Q_r\|x_N - r_N\|^2 + Q_{u_f}\|\pi_{\Xi_\pi}(r_N, r_N)\|^2.\end{aligned} \tag{19}$$

The obstacle enters through the one-sided hinge $[-h_{\mathrm{obs}}(x_t)]_+$, so the penalty grows linearly in the depth of violation. The terminal term $Q_{u_f}\|\pi_{\Xi_\pi}(r_N, r_N)\|^2$, which evaluates the policy at the target and therefore depends on Ξ_π directly, is required because without it the policy holds a residual increment at the target and the reaching task does not settle. For this example, the IPOPT benchmark instead minimizes the control effort $\sum_t \|u_t\|_1$ subject to the terminal condition $x_N = r_N$ and the obstacle constraint, both imposed as hard constraints, so that it reaches the setpoint exactly by construction.

D. Training Setup

In each case the prediction map F_Ξ is identified first by solving (5) and then held fixed while the policy coefficients are learned; both stages are sparsified by Algorithm 1. The constants for both stages are collected in Table I. The retained terms depend jointly on λ_π and τ_π; we select the pair by the sweep of Fig. 2, retaining the sparsest policy whose task performance on a separate set of validation scenarios is not degraded relative to the dense end of the sweep; the evaluation scenarios in Section V are held out from this selection. The price of sparsity is system dependent: for the Van der Pol oscillator the loss is flat across the sweep, whereas for the two-tank system it degrades once λ_π is pushed beyond the selected point.

TABLE I

IDENTIFICATION AND POLICY TRAINING PARAMETERS.

	Two-Tank	Van der Pol	2-D Int.
Identification			
Trajectories m_{id}	2000	2000	1500
Horizon N_{id}	5	3	3
Integrator	RK4	RK4	discrete
Step Δt [s]	1.0	0.1	–
Epochs n_e	20 000	6000	20 000
Weights (Q_1, Q_N)	$(1, 2)$	$(1, 2)$	$(1, 2)$
Sparsity λ	3×10^{-6}	1×10^{-5}	1×10^{-9}
Threshold τ	5×10^{-3}	1×10^{-2}	1×10^{-2}
Prune interval n_p	5000	1000	5000
Library size p	15	18	19
Policy			
Scenarios m	2000	1000	2000
Horizon N	50	50	100
Sparsity λ_π	0.1	0.001	0.016
Threshold τ_π	0.2	0.0625	0.004
Library size p_π	22	14	29
Leak ϵ	0.01	0.01	0.05
Band b	0.05	0.05	0.01
Stage weights	$Q_r = 20$ $Q_x = 20$	$Q_r = 20$, $Q_x = 20$ $Q_u = 5$, $Q_{du} = 1$	$Q_r = 1$, $Q_{dx} = 1$ $Q_u = Q_{du} = 10$ $Q_h = 100$, $Q_{u_f} = 5$

Notes: The policy epoch budget is 12 000, with the retained set pruned every $n_p = 800$ epochs. Patience is $M = 3$ throughout. Both stages use AdamW [25] under Algorithm 1 with weight decay 0.01, which remains active during the debiasing refit of $K = 800$ epochs.

V. RESULTS

Fig. 3 and Table II summarize the closed-loop evaluation. The identified models used for policy training are accurate over the prediction horizon: on held-out trajectories, their $N = 50$ step rollout mean-squared errors are 7.64×10^{-10}, 3.30×10^{-8}, and 4.71×10^{-7} for the two-tank, Van der Pol, and 2-D integrator systems, respectively, with a maximum absolute state error below 1.37×10^{-3}. Across all three

TABLE II

CONTROLLER EVALUATION ACROSS CASE STUDIES.

Method	Task loss $[10^{-3}]$	Control effort	Time [μs] / memory	Violations [%] / depth
Two-Tank				
IPOPT	8.8×10^{-6}	–	8444 / 26 MiB	0.0 / –
NN-DPC	0.144 ± 0.013	–	30 / 5.01 kB	0.0 / –
SD-DPC	$\mathbf{0.012 \pm 0.006}$	–	**17 / 0.047 kB**	0.0 / –
NN-Distilled	0.209 ± 0.039	–	17 / 0.047 kB	0.0 / –
Van der Pol				
IPOPT	4.6×10^{-24}	–	301 / 19 MiB	0.0 / –
NN-DPC	$\mathbf{41.0 \pm 124.4}$	–	29 / 5.01 kB	0.0 / –
SD-DPC	49.9 ± 126.4	–	**17 / 0.023 kB**	0.0 / –
NN-Distilled	96.8 ± 183.8	–	17 / 0.023 kB	0.0 / –
2-D Integrator				
IPOPT	0.000 ± 0.000	12.00 ± 1.18	1841 / 18 MiB	0.0 / –
NN-DPC	$\mathbf{0.298 \pm 0.560}$	$\mathbf{12.00 \pm 1.16}$	28 / 5.01 kB	0.0 / –
SD-DPC	4.75 ± 4.79	12.07 ± 1.12	**17 / 0.094 kB**	0.2 / 0.01
NN-Distilled	62.6 ± 58.0	12.07 ± 1.03	17 / 0.094 kB	60.8 / 1.61

Notes: Mean $\pm$ standard deviation over 500 held-out test scenarios simulated on the true plant; IPOPT losses are numerically zero and given as means. Horizons are 1000, 50, and 100 steps; task loss is the MSE over the final 100 steps of each 250-step reference segment, the final 10 steps, and the terminal state, respectively. Control effort is $\sum_t \|u_t\|_1$. Time is per control step; for IPOPT it is the solve time divided by N. Memory is the parameter storage for the learned policies, and the resident memory added by each solve for IPOPT. Violations give the percentage of scenarios with any state-constraint violation and the maximum violation depth, measured by $[-h_{\mathrm{obs}}(x_t)]_+$. NN-Distilled uses the terms retained by SD-DPC, fitted by unregularized least squares.

systems, the sparse policy evaluates in $17\,\mu$s and occupies under $0.1\,$kB, roughly two orders of magnitude less memory than NN-DPC and far below the 18–26 MiB allocated by each IPOPT solve. Its closed-loop behavior, however, differs from task to task.

Proposition 3 is evaluated on the levels $x \in [0.05, 1]^2$ and references $r \in [0.1, 0.9]$ for the two-tank system and, for the other two systems, on a box enclosing all evaluation trajectories. Weighting each retained coefficient by the slope of its library function, (11) certifies $L_\pi \le 18.4$, 16.0, and 0.32, whereas the largest $\|\partial_x \pi\|$, computed by automatic differentiation on a dense grid over these sets, is 12.0, 9.4, and 0.19, so the certificate is conservative by a factor of at most 1.7; the pruning bound (ii) also held at every prune of Algorithm 1.

A. Two-Tank System

For the two-tank system, SD-DPC retains six terms across the two input channels (Fig. 3(a)), tracks each reference segment, and satisfies the level constraints on every scenario. Its tracking loss is about twelve times lower than that of NN-DPC, a gap well beyond the scenario-to-scenario spread of either policy, so the sparse parameterization costs nothing in accuracy here; both learned policies remain well above the IPOPT benchmark, which optimizes each scenario online. The distilled policy, which uses the same six terms, also satisfies the constraints, but its coefficients, fitted to the actions of NN-DPC, leave a steady offset, and its tracking loss is about seventeen times that of SD-DPC.

The raw outputs of the learned SD-DPC pump and valve channels are

$$\begin{aligned} v_1 &= 0.6198\, r + 1.4659 \cos 2x_1 + 2.1415 \cos 2x_2 \\ &\quad - 3.4944 \cos 2r, \\ v_2 &= -4.2401 \cos 2x_1 + 4.2429 \cos 2x_2, \end{aligned} \tag{20}$$

with $u = \Pi_{[0,1]^2}(v)$ and r the level reference. The pump command combines a reference-dependent feedforward with feedback on both levels, while the two nearly equal valve coefficients make the valve open only when the upper tank is fuller than the lower one.

B. Forced Van der Pol Oscillator

In the Van der Pol case, all controllers stabilize the origin and satisfy the constraints on every scenario. The benchmark optimizes each scenario individually and therefore reaches a loss below 10^{-5}, whereas both learned policies settle more slowly, with a final-10-step loss of about $4\text{–}5 \times 10^{-2}$. The loss of SD-DPC is comparable to that of NN-DPC, whereas the distilled policy on the same terms reaches about twice that of SD-DPC. What the sparse parameterization adds here is structure. The policy found by SD-DPC (Fig. 3(b)) retains only three terms,

$$\begin{aligned} v(x) &= \kappa_1 x_1 + \kappa_2 x_2 + \kappa_3 x_1^2 x_2, \\ \kappa &= (-0.9873,\, -2.8860,\, 0.2716), \end{aligned} \tag{21}$$

with $u = \Pi_{[-5,5]}(v)$, and is therefore simple enough for direct analysis (Remark 1).

Remark 1. Because F_Ξ and π_{Ξ_π} are finite expansions over explicit analytic libraries, the learned closed loop is available in closed form and admits direct analysis in specific cases. For the policy (21) applied to (15) without saturation, $V = \frac{1}{2}(1-\kappa_1)x_1^2 + \frac{1}{2}x_2^2$ yields $\dot V = (\mu+\kappa_2)x_2^2 - (\mu-\kappa_3)x_1^2 x_2^2 \le 0$ whenever $\kappa_1 < 1$, $\kappa_2 < -\mu$, and $\kappa_3 \le \mu$, conditions satisfied by the learned coefficients, so that the origin of this continuous-time closed loop is globally asymptotically stable [30, Cor. 4.2].

C. Two-Dimensional Integrator with Obstacle Avoidance

This case study shows why the policy must be trained in closed loop, with the constraints enforced during optimization. SD-DPC retains twelve terms (Fig. 3(c)), steers around the obstacle, and reaches the setpoint from every sampled initial condition, with cumulative control effort within 0.6% of the minimum attained by IPOPT. SD-DPC and NN-DPC are both trained through closed-loop rollouts that include the obstacle penalty. Both satisfy the constraint, except in one SD-DPC scenario (depth 0.01). The terminal error of SD-DPC is higher than that of NN-DPC, the price of a twelve-term parameterization, remains well within the reaching task. The distilled policy violates the obstacle constraint in 60.8% of the scenarios, with a maximum violation depth of 1.61, and its terminal error is about thirteen times that of SD-DPC. Its coefficients are regressed onto the teacher's actions without the obstacle penalty, Proposition 3(iii), applied with π the student and $\tilde\pi$ the teacher on the same plant ($\delta = 0$), explains why this gives no safety guarantee. The imitation error $\|\pi(x) - \tilde\pi(x)\|$ is at most 0.08 on the states visited by the teacher, where the regression data lie, but reaches

0.35 on those visited by the student, so $\varepsilon_\pi \geq 0.35$. For Proposition 3(iii) to certify that the student inherits the teacher's constraint satisfaction, the teacher would need a margin $h_{\text{obs}} \geq L_h e_t \geq L_h L_u \varepsilon_\pi \geq 1.05$, since $L_u = 1$ for the integrator and $L_h \geq 3$ outside the obstacle, where $\|\nabla h_{\text{obs}}\| = 2\|x - x^o\| \geq 3$. The near-optimal teacher, however, clears the obstacle by only $h_{\text{obs}} = 0.06$, so no margin is left to absorb the imitation error.

VI. Conclusions

This work presented SD-DPC, a framework in which both the prediction model and the feedback policy are sparse combinations of candidate functions, learned by the same thresholded gradient-descent procedure, so that the policy terms are chosen by the constrained closed-loop objective rather than by imitating a pretrained controller. Across three case studies, the learned SD-DPC policies retained only a few terms, satisfied the constraints in almost all test scenarios, and reduced the task loss of a policy distilled onto the same terms by up to an order of magnitude, whereas the distilled policy violated the obstacle constraint. The closed-form policies require about two orders of magnitude less memory than deep neural policies and far less memory and evaluation time than the optimization-based benchmark. The sparse policies also admit explicit sensitivity bounds and, in specific cases, direct stability analysis, as shown for the Van der Pol oscillator. Constraint satisfaction is currently promoted through penalty terms and verified empirically. Future work will focus on exploiting the symbolic structure of SD-DPC for stronger closed-loop guarantees. This includes hard feasibility through safety filters, systematic region-of-attraction certification, robustness bounds under identification error, and sufficient conditions under which sparsification preserves stability and feasibility. We also plan to address noisy data and library mismatch, task-informed libraries, and online adaptation of the retained coefficients after deployment.

References

[1] C. E. Garcia, D. M. Prett, and M. Morari, "Model predictive control: Theory and practice—a survey," *Automatica*, vol. 25, no. 3, pp. 335–348, 1989.

[2] D. Q. Mayne, J. B. Rawlings, C. V. Rao, and P. O. M. Scokaert, "Constrained model predictive control: Stability and optimality," *Automatica*, vol. 36, no. 6, pp. 789–814, 2000.

[3] M. Schwenzer, M. Ay, T. Bergs, and D. Abel, "Review on model predictive control: An engineering perspective," *The international journal of advanced manufacturing technology*, vol. 117, no. 5, pp. 1327–1349, 2021.

[4] A. Bemporad, M. Morari, V. Dua, and E. N. Pistikopoulos, "The explicit linear quadratic regulator for constrained systems," *Automatica*, vol. 38, no. 1, pp. 3–20, 2002.

[5] M. Hertneck, J. Köhler, S. Trimpe, and F. Allgöwer, "Learning an approximate model predictive controller with guarantees," *IEEE Control Syst. Lett.*, vol. 2, no. 3, pp. 543–548, 2018.

[6] B. Karg and S. Lucia, "Efficient representation and approximation of model predictive control laws via deep learning," *IEEE Trans. Cybern.*, vol. 50, no. 9, pp. 3866–3878, 2020.

[7] Y. Li, K. Hua, and Y. Cao, "Using stochastic programming to train neural network approximation of nonlinear MPC laws," *Automatica*, vol. 146, p. 110665, 2022.

[8] J. Drgoňa, K. Kiš, A. Tuor, D. Vrabie, and M. Klaučo, "Differentiable predictive control: Deep learning alternative to explicit model predictive control for unknown nonlinear systems," *J. Process Control*, vol. 116, pp. 80–92, 2022.

[9] J. Drgoňa, A. Tuor, and D. Vrabie, "Learning constrained parametric differentiable predictive control policies with guarantees," *IEEE Trans. Syst., Man, Cybern. Syst.*, vol. 54, no. 6, pp. 3596–3607, 2024.

[10] H. Hose, A. Gräfe, and S. Trimpe, "Parameter-adaptive approximate MPC: Tuning neural-network controllers without retraining," in *Proc. 6th Annual Learning for Dynamics & Control Conference*, ser. Proceedings of Machine Learning Research, vol. 242, 2024, pp. 349–360.

[11] S. L. Brunton, J. L. Proctor, and J. N. Kutz, "Discovering governing equations from data by sparse identification of nonlinear dynamical systems," *Proc. Natl. Acad. Sci. USA*, vol. 113, no. 15, pp. 3932–3937, 2016.

[12] S. L. Brunton, J. L. Proctor, and J. N. Kutz, "Sparse identification of nonlinear dynamics with control (SINDYc)," *IFAC-PapersOnLine*, vol. 49, no. 18, pp. 710–715, 2016.

[13] E. Kaiser, J. N. Kutz, and S. L. Brunton, "Sparse identification of nonlinear dynamics for model predictive control in the low-data limit," *Proc. R. Soc. A*, vol. 474, no. 2219, p. 20180335, 2018.

[14] D. A. Messenger and D. M. Bortz, "Weak SINDy: Galerkin-based data-driven model selection," *Multiscale Model. Simul.*, vol. 19, no. 3, pp. 1474–1497, 2021.

[15] U. Fasel, J. N. Kutz, B. W. Brunton, and S. L. Brunton, "Ensemble-SINDy: Robust sparse model discovery in the low-data, high-noise limit, with active learning and control," *Proc. R. Soc. A*, vol. 478, no. 2260, p. 20210904, 2022.

[16] K. Champion, B. Lusch, J. N. Kutz, and S. L. Brunton, "Data-driven discovery of coordinates and governing equations," *Proc. Natl. Acad. Sci. USA*, vol. 116, no. 45, pp. 22 445–22 451, 2019.

[17] K. Kaheman, S. L. Brunton, and J. Nathan Kutz, "Automatic differentiation to simultaneously identify nonlinear dynamics and extract noise probability distributions from data," *Machine Learning: Science and Technology*, vol. 3, no. 1, p. 015031, 2022.

[18] L. Zhang, S. Tang, and G. He, "Learning chaotic systems from noisy data via multi-step optimization and adaptive training," *Chaos: An Interdisciplinary Journal of Nonlinear Science*, vol. 32, no. 12, 2022.

[19] M. Landajuela, B. K. Petersen, S. Kim, C. P. Santiago, R. Glatt, N. Mundhenk, J. F. Pettit, and D. Faissol, "Discovering symbolic policies with deep reinforcement learning," in *International Conference on Machine Learning*. PMLR, 2021, pp. 5979–5989.

[20] J. Guo, R. Zhang, S. Peng, Q. Yi, X. Hu, R. Chen, Z. Du, X. Zhang, L. Li, Q. Guo, and Y. Chen, "Efficient symbolic policy learning with differentiable symbolic expression," in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 36 278–36 304.

[21] N. Botteghi and U. Fasel, "Parametric PDE control with deep reinforcement learning and ℓ_0 sparse polynomial policies," in *Proc. IEEE 63rd Conference on Decision and Control (CDC)*, 2024, pp. 6108–6115.

[22] N. Zolman, C. Lagemann, U. Fasel, J. N. Kutz, and S. L. Brunton, "SINDy-RL for interpretable and efficient model-based reinforcement learning," *Nat. Commun.*, vol. 16, p. 10714, 2025.

[23] K. Kunisch, D. Vásquez-Varas, and D. Walter, "Learning optimal feedback operators and their sparse polynomial approximations," *Journal of Machine Learning Research*, vol. 24, no. 301, pp. 1–38, 2023.

[24] Y. Bengio, N. Léonard, and A. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," arXiv preprint arXiv:1308.3432, 2013. [Online]. Available: https://arxiv.org/abs/1308.3432

[25] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," *arXiv preprint arXiv:1711.05101*, 2017.

[26] M. Fazlyab, A. Robey, H. Hassani, M. Morari, and G. Pappas, "Efficient and accurate estimation of lipschitz constants for deep neural networks," *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[27] J. Drgona, A. Tuor, J. Koch, M. Shapiro, B. Jacob, and D. Vrabie, "NeuroMANCER: Neural Modules with Adaptive Nonlinear Constraints and Efficient Regularizations," 2023. [Online]. Available: https://github.com/pnnl/neuromancer

[28] A. Wächter and L. T. Biegler, "On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming," *Math. Program.*, vol. 106, no. 1, pp. 25–57, 2006.

[29] J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl, "CasADi – A software framework for nonlinear optimization and optimal control," *Mathematical Programming Computation*, vol. 11, no. 1, pp. 1–36, 2019.

[30] H. K. Khalil, *Nonlinear Systems*, 3rd ed. Upper Saddle River, NJ: Prentice Hall, 2002.